Project Jupyter Tutorial Report

Creating a Pandas dataframe on films via webscrapping then Lite-EDA

Introduction

This document covers a wide variety of useful material for one delving into manual data analysis via python and a simple computer; it also covers all the material and more from the video series tutorial on data science. All the technical details are presented below in an orderly fashion, with the conceptual explanations and review first and the coding review at the end.

Topics touched upon in this report include:

- Setting up Python on Windows and starting out with some of the right libraries
- Running a Jupyter Notebook in the browser to work on the script
- Importing, scraping, and compiling the dataset
- Cleaning and organizing the dataset
- Presenting and analyzing the data

Initially, only a Kaggle dataset was going to be reviewed, but having a desire for larger and more robust data led to the scraping of data from two sites with python. A very simple EDA (exploratory data analysis) was performed on the Kaggle dataset followed by another light but slightly juicier EDA for the scraped dataset.

The Kaggle Data Set

The first dataset that was used was sourced from Kaggle.com; searching Google for a dataset led there, and signing into a free account on their site allows one to download a .zip file from a library of datasets. It was then extracted and called into the jupyter notebook and set up as a Pandas dataframe for analysis.

The dataframe consisted of 5,000 rows and 13 features:

'Rank', 'Title', 'Worldwide', 'Domestic', 'Domestic %',
'$Foreign', 'Foreign %', 'Year', 'Genres',
'Rating', 'Vote_Count', 'Original_Language',
'Production_Countries', 'Rating_clean'

The data spanned films released between 2000 to 2024 hence its EDA gives insights only for that scope of time.

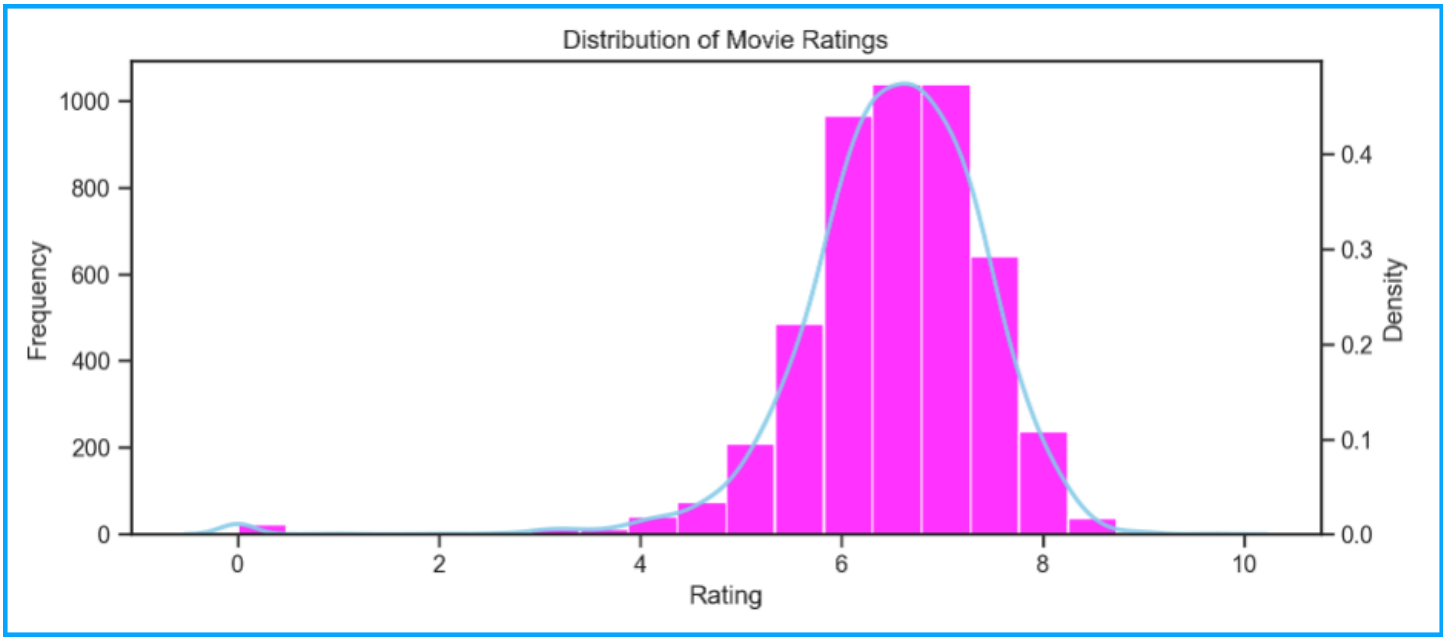
Light-EDA:

Top 5 Horror (*table 1*) :

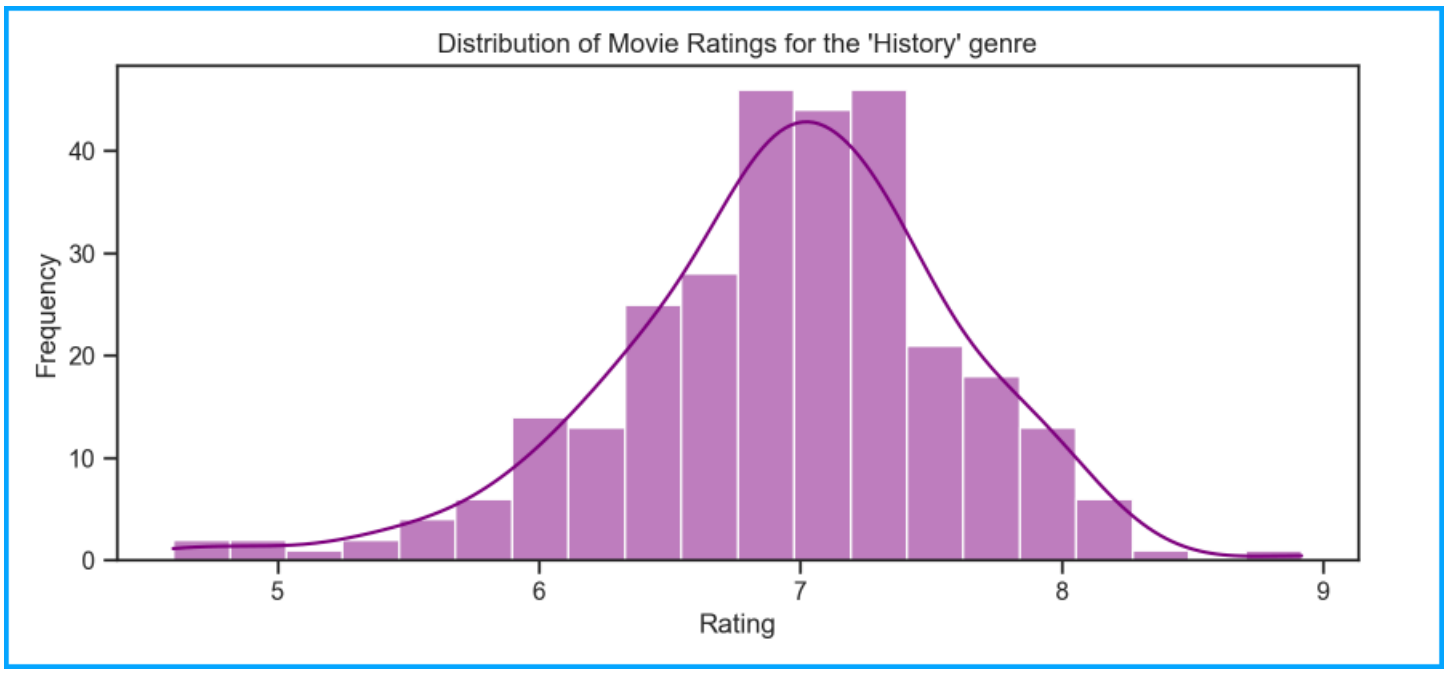
[6]:

	Title	Year	$Worldwide	$Domestic	$Foreign	Genres
1	It	2017	$702,781,748	$327,481,748	$375,300,000	Horror, Thriller
2	World War Z	2013	$540,007,876	$202,359,711	$337,648,165	Action, Horror, Science Fiction, Thriller
3	The Meg	2018	$529,243,742	$145,443,742	$383,800,000	Action, Science Fiction, Horror
4	It Chapter Two	2019	$473,093,228	$211,593,228	$261,500,000	Horror, Drama, Thriller
5	Beetlejuice Beetlejuice	2024	$451,100,435	$294,100,435	$157,000,000	Comedy, Fantasy, Horror

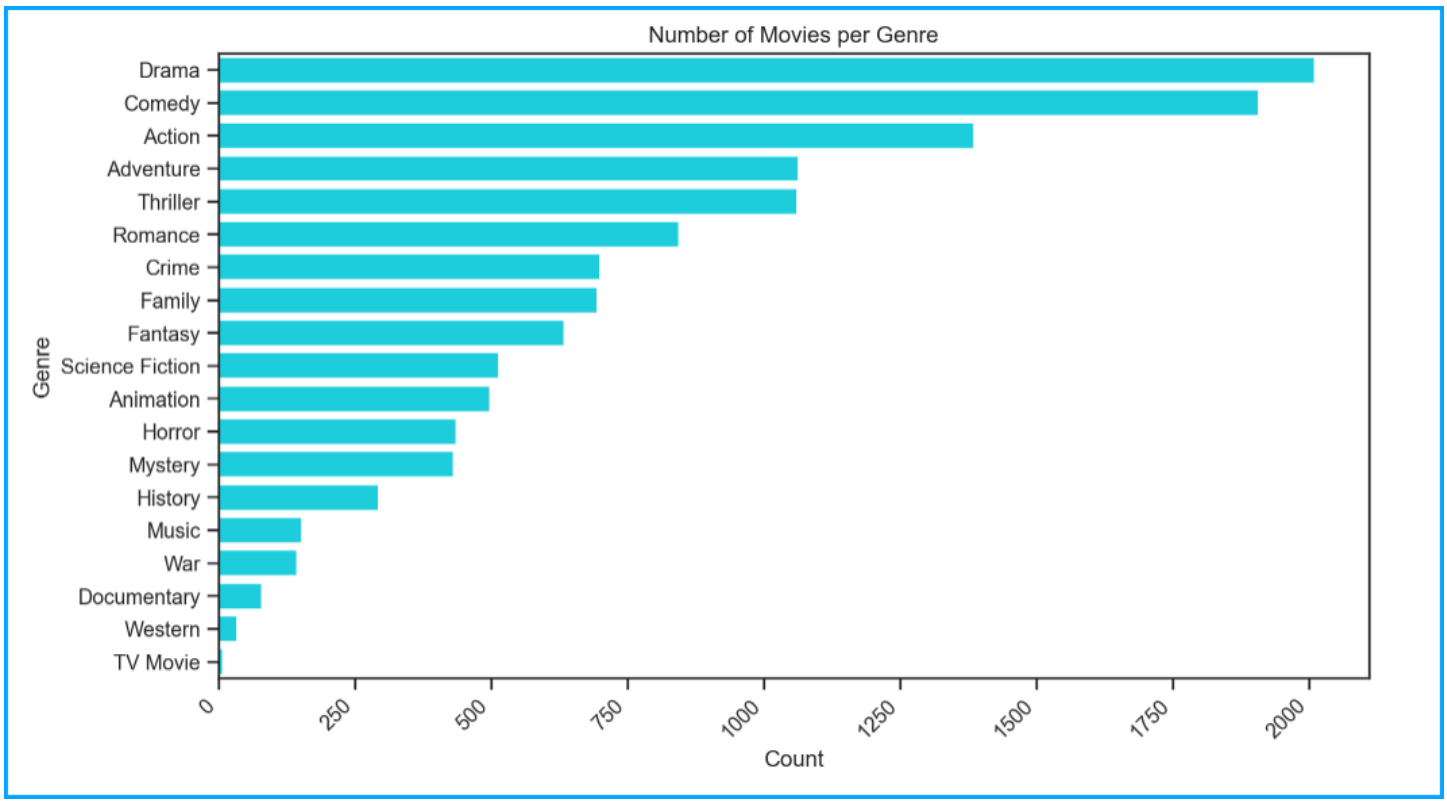
Histogram (*chart 1*) showing amount of movies per their IMDb ratings with KDE line added:



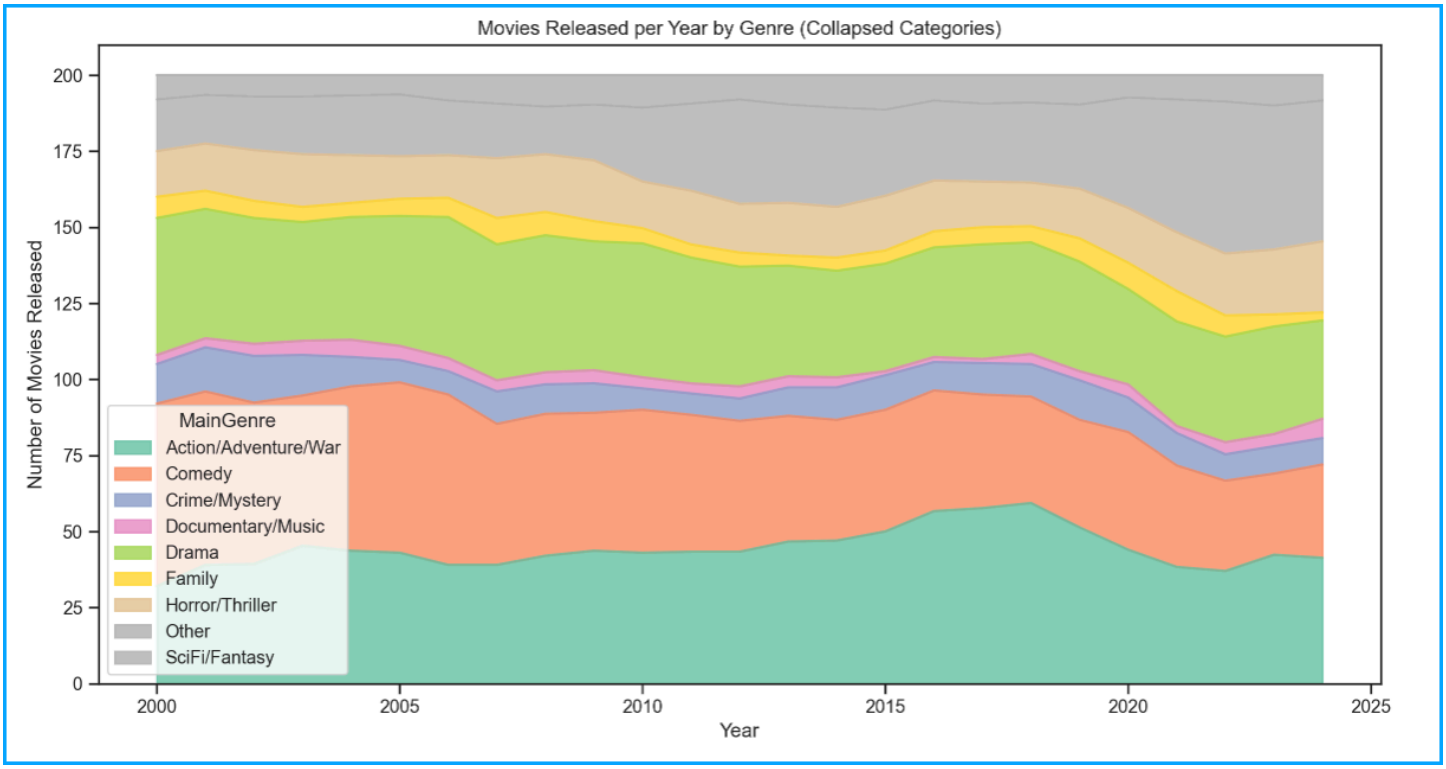
Another histogram (*chart 2*) showing rating frequencies again but now specifically for ‘History’ movies:



Count plot (*chart 3*) showing amount of movies per genre (same movie can be in multiple genres):



Stacked area plot (*chart 4*) showing amount of movies released per genre per year; this time the 'Genre' feature was consolidated to have one genre per movie:



The Scraped Data Set

In desire of a more robust dataset, it was decided to scrape one together. Two different scraping techniques were used to pull data from BoxOfficeMojo.com and OMDbAPI.com, both reputable sources. After scraping from BoxOfficeMojo 9,252 records of data were harnessed with 8 features. Looking to expand the amount of features for the same amount of films, OMDb was also scraped. OMDb's API only allows for 1,000 pull requests per day for free users, so that original scraped data was broken into 10 blocks of up to 1,000 records each. Each day a block of data was used as the reference for which movie to look up from OMDb and after 10 days all of it concatenated together.

Compared to the Kaggle dataset, there were 25 more years of film data compared to the original 24 years (now from 1977 to 2026); this entails 4,252 more films paired with 17 more features. There were additional columns that were dropped due to the amount of missing values as well as irrelevancy.

The final compiled dataframe consisted of 9,252 rows and the following 30 features :

'Total_rank',	'Title',	'Year',	'Yearly Rank',	'Worldwide',
'Domestic',	'D%',	'Foreign',	'F%',	'WRev',
'DRev',	'FRev',	'Rated',	'Released',	'Runtime_min',
'Genre',	'Director',	'Writer',	'Actors',	'Plot',
'Language',	'Country',	'Awards',	'Poster',	'Metascore',
'imdbRating',	'imdbVotes',	'imdbID',	'BoxOffice',	'RTomatoes'

Forming the final dataframe:

- Dataframe sorted by 'WRev' descending and has 'Total_rank' as its index
- 'Released' had its values cleaned then parsed as datetime dtype
- Columns dropped: 'DVD', 'Production', 'Website', 'Total_seasons', 'Ratings', 'Response', & 'Type'
- Formatting and checking syntax and dtypes of all the columns

Light-EDA:

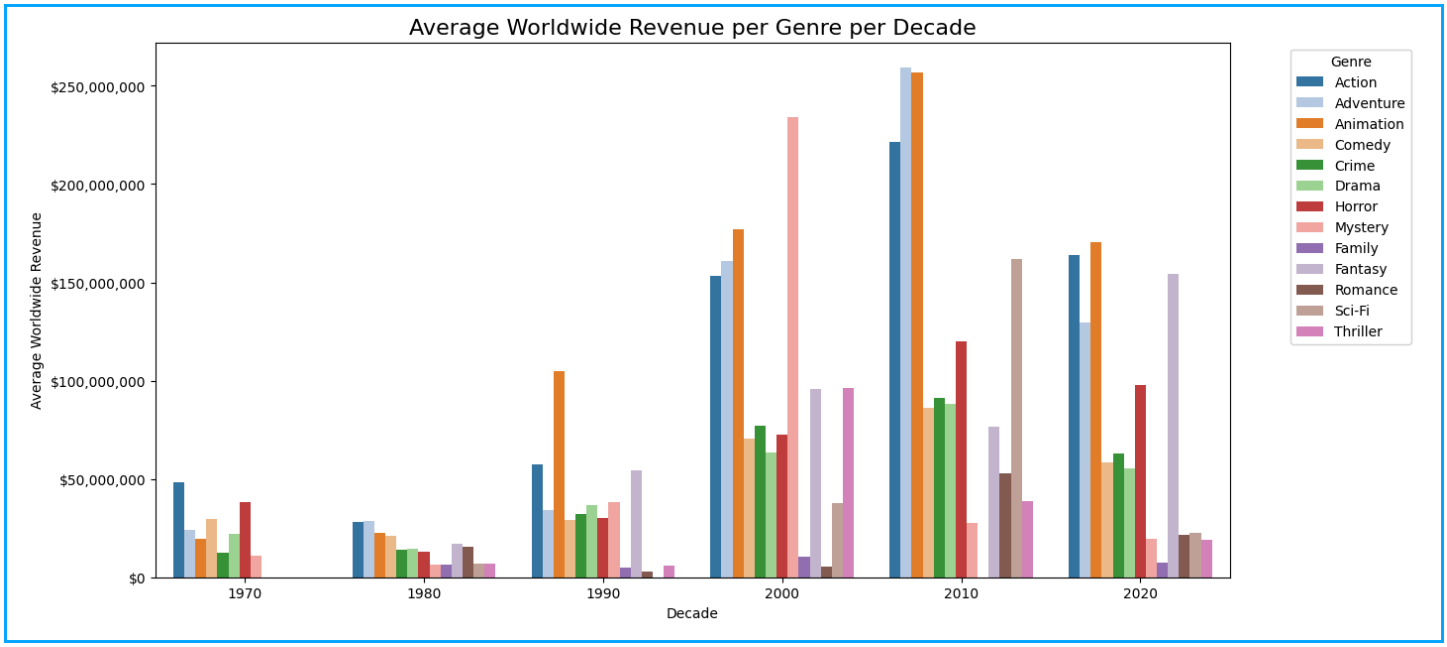
Top 5 Horror (*table 2*):

[31]:

	Title	Year	Worldwide	Domestic	Foreign	Genre
1	It	2017	$702,781,748	$327,481,748	$375,300,000	Horror
2	I Am Legend	2007	$585,410,052	$256,393,010	$329,017,042	Drama, Horror, Sci-Fi
3	World War Z	2013	$540,007,876	$202,359,711	$337,648,165	Action, Adventure, Horror
4	The Meg	2018	$529,243,742	$145,443,742	$383,800,000	Action, Adventure, Horror
5	The Conjuring: Last Rites	2025	$499,256,445	$177,756,445	$321,500,000	Horror, Mystery, Thriller

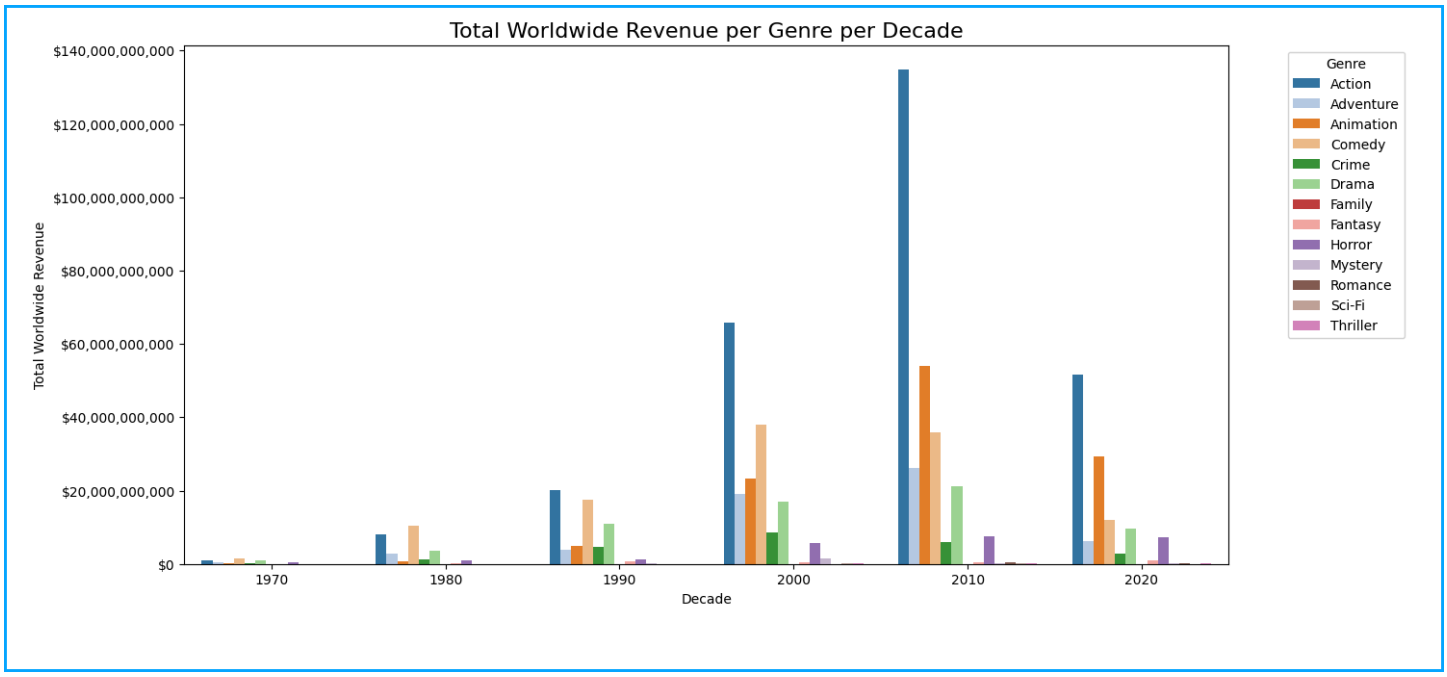
- 'I Am Legend' did not appear in the Kaggle 'Top 5 Horror' since it had its genres categorized differently in its dataset

Bar plot (*chart 5*) showing 'Average Worldwide Revenue per Genre per Decade':

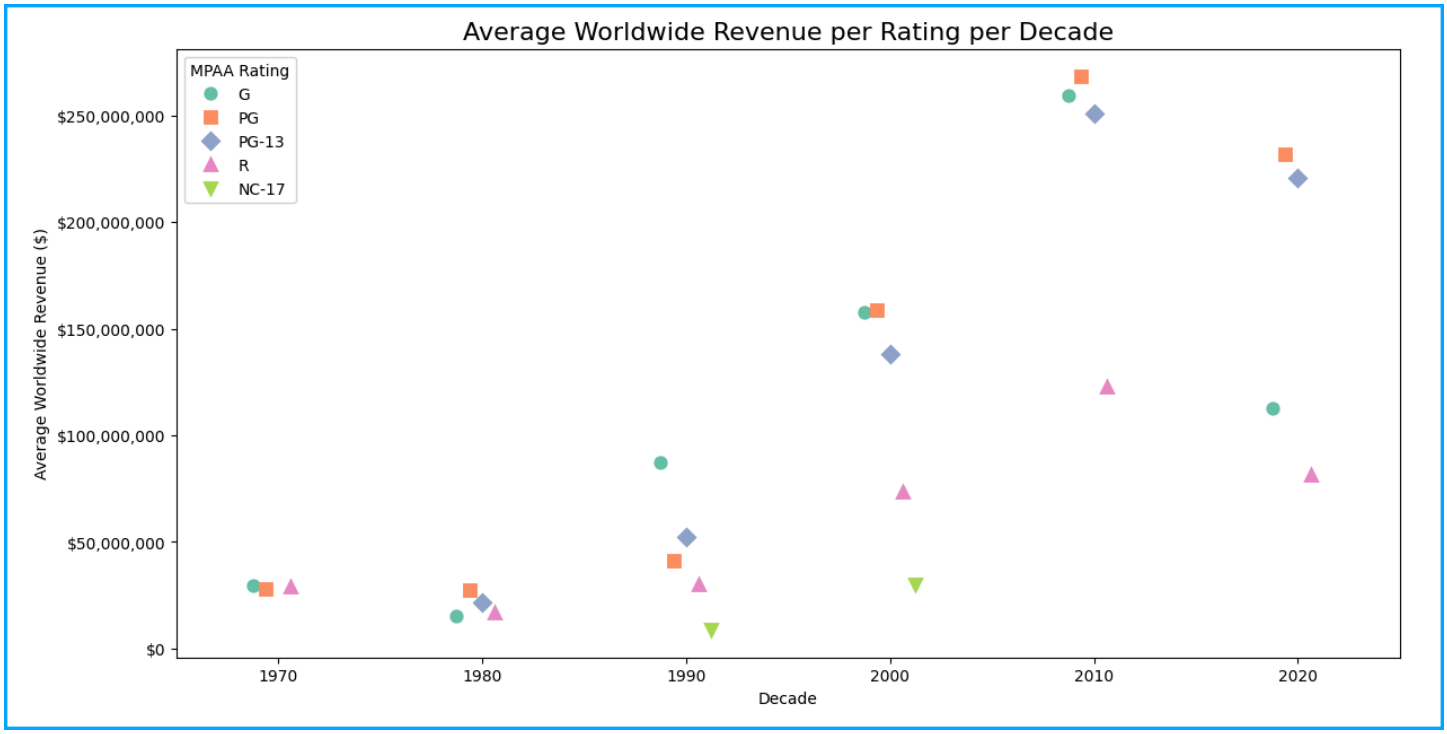


- Most of the genres generated increased revenues each decade except for a few exceptions that can be seen in the 2010's (i.e. 'Family' and 'Fantasy' genres)

Bar plot (*chart 6*) showing 'Total Worldwide Revenue per Genre per Decade':

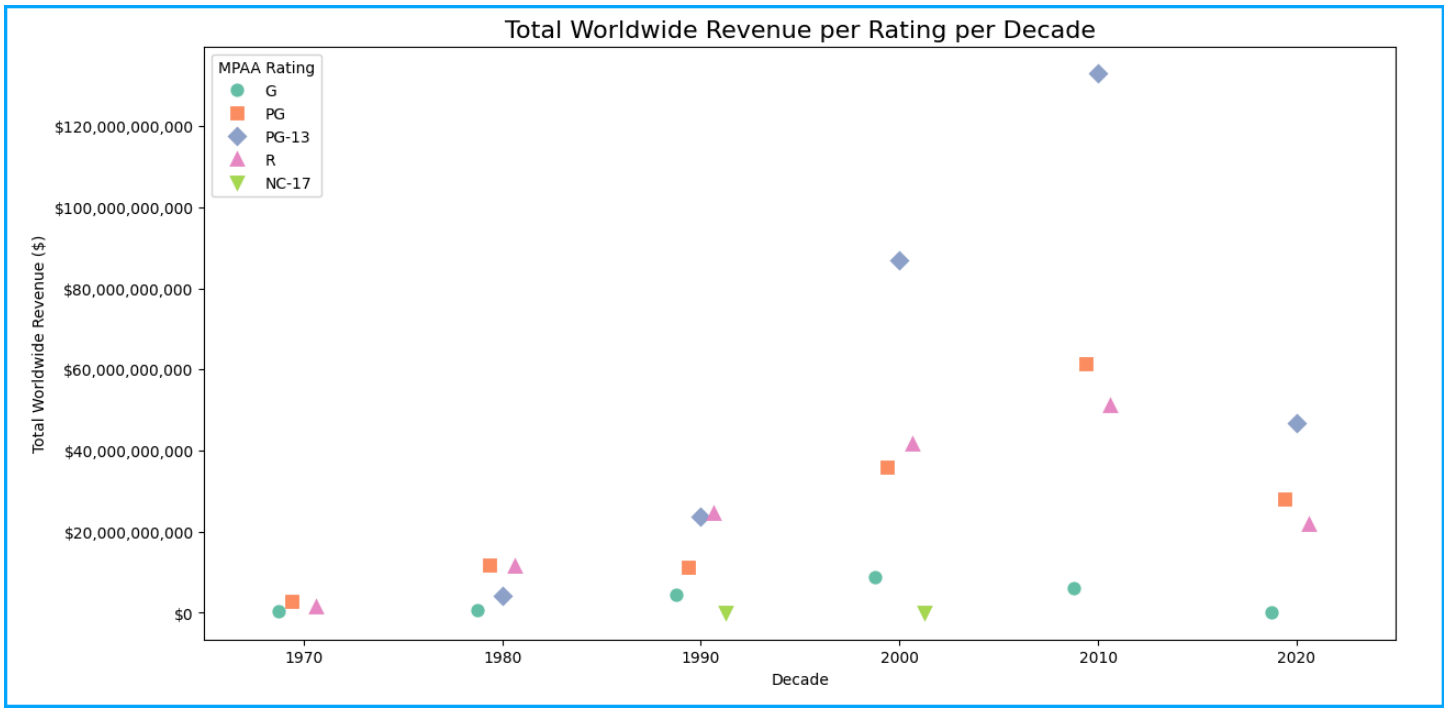


Point plot (*chart 7*) showing 'Average Worldwide Revenue per Rating per Decade':



- It appears that films targeting the younger audiences have been generating more revenue than movies rated ‘R’ or ‘NC-17’

Point plot (*chart 8*) showing 'Total Worldwide Revenue per Rating per Decade':



- The 90's had rated 'R' movies making the most but the following decades show incredible performances from movies rated 'PG-13'

Conclusion

This project provided a fortification for understanding some basics regarding data engineering and data analysis. Being a personal project, it is not apparent whether this section or other parts of the project will be reviewed by others; regarding any feedback, positive or negative, please feel free to reach out.

After first exploring a dataset from Kaggle, it was understood that it is crucial to use reliable data sources. Although the person who uploaded the data to Kaggle followed a slightly similar process, the results and robustness of the data were questionable. For example, the 'production budget' feature was mentioned on Kaggle but did not appear in the data, sorting and reranking was required, and more.

Scraping the data provided a more reliable source and allowed fine-tuning of which data to acquire. Once the final dataframe was compiled, it provided a stronger base to perform analysis and extract interesting insights.

The last section of this report contains practically all the code used for this project and can serve as a useful reference for projects requiring similar components in their scripts.

In the future, the final compiled dataframe may be uploaded to Kaggle for others to use. Further analysis can be conducted with machine learning models implemented to also provide predictive insights such as predicted revenue or ratings for future films based on variable derived from different features (ie. 'Genre', 'Plot', 'Actors', 'Director', etc).

Coding Review

Windows Command Prompt - CMD Syntax

Setting up virtual python environment (after installing):

```
-m venv {given name}
```

Activating python environment:

```
{given name}\Scripts\activate
```

Installing python packages:

```
pip install numpy pandas scipy scikit-learn seaborn matplotlib jupyterlab
```

Review versions:

```
python --version
&  pip list
```

Open jupyter notebook:

```
jupyter lab
```

Jupyter Notebook Initial Setup - Python

Initial import of base packages that will be used:

```
import pandas as pd, numpy as np, matplotlib.pyplot as plt, seaborn as sns
```

Importing a spreadsheet file as a pandas dataframe from one's local machine:

```
df = pd.read_csv(r"C:\{directory path here}.csv")
```

Initial overview of the dataframe, showing amount of records/features/nulls and more:

```
df.info()
```

To show all columns when doing .head() :

```
pd.set_option('display.max_columns', None)
```

Gives a glimpse of the top of the dataframe (top 5 rows by default):

```
df.head()
```

Scraping Box Office Mojo Site

Import of necessary python packages for scraping:

```
import requests, time
from bs4 import BeautifulSoup
```

Setting up of web addresses to be scraped:

```
BASE_URL = "https://www.boxofficemojo.com/year/world/{}/"
years = range(1977, 2027)
```

A fresh list that will have the scraped data appended to it:

```
all_rows = []
```

‘For’ loop that will iterate the scraping for each web address:

```
for year in years:
    url = BASE_URL.format(year)
    print(f"Scraping {year}...")

    response = requests.get(url)
    soup = BeautifulSoup(response.text, "html.parser")

    table = soup.find("table")
    if table is None:
        print(f"! No table found for {year}")
        continue

    rows = table.find_all("tr")

    # Skip header row
    for row in rows[1:]:
        cols = [c.get_text(strip=True) for c in row.find_all("td")]
        if len(cols) == 7:
            all_rows.append([year] + cols)

    time.sleep(1)  # be polite to the server
```

Converting the data scraped from list to a Pandas dataframe:

```
columns = [ "Year",  "Rank",   "Release Group",   "Worldwide",
      "Domestic", "Domestic %", "Foreign",   "Foreign %"]
df = pd.DataFrame(all_rows, columns=columns)
```

Exporting a backup record of the data as a CSV:

```
df.to_csv("box_office_77_26.csv", index=False)
```

Scraping OMDb via their API

Import of necessary python packages for scraping:

```
import requests, time, pickle, OS
```

Initial setup for accessing the API and having a cache file:

```
API_KEY = "**insert here the key OMDb sends you**"
BASE_URL = "http://www.omdbapi.com/"
CACHE_FILE = "omdb_cache.pkl"
BATCH_START = 0000
BATCH_SIZE = 1000   # based on the OMDb limit for free users
start = BATCH_START
end = BATCH_START + BATCH_SIZE
df_block = df.iloc[start:end].copy()
```

Scraping 1000 records from OMDb with an initial sample and ongoing progress updates:

```
    print(f"Processing batch from {start} to {end} (size: {len(df_block)})")

    def get_movie_data(title, year):
        params = { "t": title, "y": year,  "apikey": API_KEY  }
        r = requests.get(BASE_URL, params=params).json()
        return r

    if os.path.exists(CACHE_FILE):
        with open(CACHE_FILE, "rb") as f:
            cache = pickle.load(f)
        print(f"Loaded cache with {len(cache)} entries.")
    else:
        cache = {}
        print("No existing cache found. Starting fresh.")

    movie_metadata = []
    success_count = 0

    for idx, row in df_block.iterrows():
        title = row["Title"]
        year = row["Year"]
        key = (title, year)
        try:
            data = get_movie_data(title, year)
            if data.get("Response") == "False":
                cache[key] = None
            else:
                cache[key] = data
                movie_metadata.append(data)
                success_count += 1
           # EARLY SAMPLE AFTER 5 SUCCESSFUL RESULTS
                if success_count == 5:
                    sample_df = pd.DataFrame(movie_metadata)
                    cols = ["Title", "Year", "Director", "Actors", "Runtime",
                            "Country", "Language", "Budget", "BoxOffice"]
                    existing_cols = [c for c in cols if c in sample_df.columns]
                    print("\n===== SAMPLE OF FIRST 5 OMDb RESULTS =====")
                    print(sample_df[existing_cols].head())
        except Exception as e:
            print(f"Error fetching {title} ({year}): {e}")
            cache[key] = None

        # Save progress every 100 movies in this batch
        if success_count % 100 == 0:
            with open(CACHE_FILE, "wb") as f:
                pickle.dump(cache, f)
            print(f"Saved after {success_count} successful movies in this batch.")

        time.sleep(0.9)  # polite delay
```

Organizing the scraped data as a dataframe:

```
with open(CACHE_FILE, "wb") as f:
    pickle.dump(cache, f)
print("Scraping complete for this batch. Building dataframe...")
clean_data = [x for x in movie_metadata if x is not None]
omdb_df = pd.DataFrame(clean_data)
```

Converting a feature of the data to integer datatype:

```
omdb_df["Year"] = pd.to_numeric(omdb_df["Year"],errors="coerce"
       ).astype("Int64")
```

Exporting a record of the data block as a CSV:

```
omdb_df.to_csv(f"merged_batchFIX_{start}_{end}.csv", index=False)
print(f"Saved merged batch to merged_batch_{start}_{end}.csv")
```

Data Cleaning and Organizing

Reviewing null records:

```
import missingno as msno

plt.ion()
plt.figure(figsize = (2,7))
msno.bar(cdf)
plt.title("Visualization of Missing Data")
plt.show()

msno.matrix(cdf)
plt.title("Display of Missing Data")
plt.show()
```

Dropping columns:

```
df = df.drop(columns=['totalSeasons','Website','Production','DVD', 'Type'])
```

Renaming features:

```
df = df.rename(columns={"Release Group": "Title"})
```

Formatting records syntax/dtype:

```
df["Rating"] = df["Rating"].str.replace("/10","",regex=False).astype(float)
df[$Worldwide] = df[$Worldwide].map("${:,.0f}".format)
```

Sorting values:

```
df = df.sort_values("WRev",ascending=False)
```

Importing all the scraped data blocks and then combining them:

```
df1 = pd.read_csv("C:/…/merged_batch_0_1000.csv",parse_dates=['Released'])
df2-9 = "                                                                "
df10 =pd.read_csv("C:/…/merged_batch_9000_10000.csv",parse_dates=['Released'])
group = [df1, df2, df3, df4, df5, df6, df7, df8, df9, df10]
films = pd.concat(group, ignore_index=True)
```

Data Presentation and Analysis

Code snippet to double the resolution of the following matplotlib plots:

```
%config InlineBackend.figure_format = 'retina'
```

New column based on the Genre feature:

```
df['Genre_clean'] = df['Genre'].str.split(',').str[0]
drop_genres =
      ['History', 'Western', 'Short', 'Music', 'Documentary', 'Biography']
df = df[~df['Genre_clean'].isin(drop_genres)]
```

Dataframe of top 5 horror movies (after sorting by worldwide revenue) (*tables 1* & 2):

```
top5_Horror = (df[df["Genres"].str.contains("Horror", na=False)]
      .loc[:, ["Title", "Year", “Worldwide”, “Domestic", "$Foreign",
      "Genres"]].head().reset_index(drop=True))
top5_Horror.index = top5_Horror.index + 1
```

Graph for ‘Distribution of Movie Ratings” *(chart 1)*:

```
fig, ax1 = plt.subplots(figsize=(10,4))
sns.histplot(df["Rating_clean"], bins=20, kde=False, color="#ff00ff",
      alpha=0.8, ax=ax1)
ax1.set_xlabel("Rating")
ax1.set_ylabel("Frequency")
ax2 = ax1.twinx()
kde_line = sns.kdeplot(df["Rating_clean"], fill=False, color="skyblue",
      alpha=0.9, linewidth=2, ax=ax2)
ax2.set_ylabel("Density")
plt.title("Distribution of Movie Ratings")
plt.show()
```

Graph for “Distribution of Movie Ratings for the 'History’ genre” *(chart 2)*:

```
dfHIS = df[df["Genres"].str.contains("History", na=False)]
plt.figure(figsize=(10,4))
sns.histplot(dfHIS["Rating_clean"], bins=20, kde=True, color="purple")
plt.title("Distribution of Movie Ratings for the 'History' genre")
plt.xlabel("Rating")
plt.ylabel("Frequency")
plt.show()
```

Graph for “Number of movies per Genre” *(chart 3)*:

```
df["Genres"] = df["Genres"].str.split(",")
df_exploded = df.explode("Genres")
df_exploded["Genres"] = df_exploded["Genres"].str.strip()
genre_counts = df_exploded["Genres"].value_counts()
genre_df = genre_counts.reset_index()
genre_df.columns = ["Genre", "Count"]

fig, ax = plt.subplots(figsize=(11,6))
sns.barplot(data=genre_df,y="Genre",x="Count", color="#00eaff",
      legend=False, ax=ax)
plt.xticks(rotation=45, ha="right")
plt.title("Number of Movies per Genre")
plt.xlabel("Count")
plt.ylabel("Genre")
plt.show()
```

Graph for “Number of movies per Genre” *(chart 4)*:

```
df2["MainGenre"] = df2.apply(assign_main_genre, axis=1)
df_year = (df2.groupby(["Year",
      "MainGenre"]).size().reset_index(name="Count"))
df_pivot = df_year.pivot(index="Year", columns="MainGenre",
       values="Count").fillna(0)
df_pivot_smooth = df_pivot.rolling(window=3, min_periods=1).mean()

plt.figure(figsize=(14,7))
df_pivot_smooth.plot.area(stacked=True,figsize=(14,7),
      colormap="Set2",alpha=0.8)
plt.title("Movies Released per Year by Genre (Collapsed Categories)")
plt.xlabel("Year")
plt.ylabel("Number of Movies Released")
plt.show()
```

Graph for ‘Average Revenue per Genre per Decade” *(chart 5)*:

```
import matplotlib.ticker as ticker
df['Decade'] = (df['Year'] // 10) * 10
avg_rev = (df.groupby(['Decade', 'Genre_clean'])['WRev'].mean().reset_index())
plt.figure(figsize=(14, 7))
palette = sns.color_palette("tab20", n_colors=df['Genre_clean'].nunique())
sns.barplot(data=avg_rev,x='Decade',y='WRev',hue='Genre_clean',
      palette=palette)
plt.title('Average Worldwide Revenue per Genre per Decade', fontsize=16)
plt.ylabel('Average Worldwide Revenue')
plt.xlabel('Decade')
plt.legend(title='Genre', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.gca().yaxis.set_major_formatter(ticker.FuncFormatter(lambda x,
      pos: f'${x:,.0f}'))
plt.show()
```

Graph for ‘Total Revenue per Genre per Decade” *(chart 6)*:

```
total_rev = (df.groupby(['Decade','Genre_clean'])['WRev'].sum().reset_index())
pivot_rev =
      total_rev.pivot(index='Decade',columns='Genre_clean',values='WRev')
pivot_rev = pivot_rev.fillna(0).sort_index()
plot_data = pivot_rev.reset_index().melt(id_vars='Decade', var_name='Genre',
       value_name='Revenue')
plt.figure(figsize=(14, 7))
sns.barplot(data=plot_data,x='Decade', y='Revenue', hue='Genre',
      palette=palette)
plt.title('Total Worldwide Revenue per Genre per Decade', fontsize=16)
plt.ylabel('Total Worldwide Revenue')
plt.xlabel('Decade')
plt.legend(title='Genre', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.gca().yaxis.set_major_formatter(ticker.FuncFormatter(lambda x,
      pos: f'${x:,.0f}'))
plt.show()
```

Graph for ‘Average Revenue per Rating per Decade” *(chart 7)*:

```
valid_ratings = ['G', 'PG', 'PG-13', 'R', 'NC-17']
df_rating = df[df['Rated'].isin(valid_ratings)].copy()
avg_rating_decade = (df_rating.groupby(['Decade',
       'Rated'])['WRev'].mean().reset_index())
plt.figure(figsize=(14, 7))
sns.pointplot(data=avg_rating_decade,x='Decade',y='WRev',hue='Rated',
      hue_order=valid_ratings,palette='Set2',dodge=0.25,
      markers=['o', 's', 'D', '^', 'v'],linestyles=[''] * 5)
plt.title('Average Worldwide Revenue per Rating per Decade', fontsize=16)
plt.xlabel('Decade')
plt.ylabel('Average Worldwide Revenue ($)')
plt.gca().yaxis.set_major_formatter(ticker.FuncFormatter(lambda x,
      pos: f'${x:,.0f}'))
plt.legend(title='MPAA Rating', loc='upper left')
plt.show()
```

Graph for ‘Total Revenue per Rating per Decade” *(chart 8)*:

```
total_rating_decade =
      (df_rating.groupby(['Decade','Rated'])['WRev'].sum().reset_index())
plt.figure(figsize=(14, 7))
sns.pointplot(data=total_rating_decade,x='Decade',y='WRev',hue='Rated',
      hue_order=valid_ratings,palette='Set2',dodge=0.25,
      markers=['o', 's', 'D', '^', 'v'],linestyles=[''] * 5)
plt.title('Total Worldwide Revenue per Rating per Decade', fontsize=16)
plt.xlabel('Decade')
plt.ylabel('Total Worldwide Revenue ($)')
plt.gca().yaxis.set_major_formatter(ticker.FuncFormatter(lambda x,
      pos: f'${x:,.0f}'))
plt.legend(title='MPAA Rating', loc='upper left')
plt.show()
```